CHAPTER 35

Tool Chaining and Orchestration

In this chapter

Introduction: Seven Calls, Four Turns, and a Metric That Lied . . . 3
35.1 Core Concepts . . . 4
35.2 A Chain Is a Graph . . . 6
35.3 Four Ways to Execute the Same Graph . . . 6
35.4 What a Chain Does to the Context Window . . . 7
35.5 Tool Search: Discovery as Retrieval . . . 8
35.6 What You Send Back After a Search . . . 9
35.7 You Can Write the Search Yourself . . . 10
35.8 Who Runs the Loop . . . 11
35.9 The Documented Tool Combinations . . . 11
35.10 Where the Loops Are . . . 12
35.11 Analogies, and Where They Break . . . 12
35.12 Architecture: Where the Decisions Live . . . 13
35.13 Worked Example: Reading a Chain Off a Transcript . . . 14
35.14 Code Walkthrough: Levels, and Why the Planner Is Six Lines . . . 15
35.15 Mini Project: A Chain, Priced Four Ways . . . 16
35.16 Experiment One: Four Ways to Run Seven Calls . . . 17
35.17 Experiment Two: A Thirty-Turn Chain . . . 17
35.18 Experiment Three: What Deferring Actually Costs . . . 18
35.19 Experiment Four: The Tool Runner's Default, Scored . . . 19
35.20 Hands-on Labs . . . 20
35.21 Lab Solutions . . . 21
35.22 Best Practices . . . 22
35.23 Common Mistakes . . . 23
35.24 Performance Considerations . . . 25
35.25 Security Considerations . . . 26
35.26 Real Production Examples . . . 26
35.27 Certification Notes . . . 27
35.28 Twenty Practice Questions . . . 28

35.29 Answers 29
35.30 Ten Practical Exercises 30
35.31 Five Challenge Labs 31
35.32 Ten Interview Questions 31
35.33 Summary 31
35.34 Glossary 33
35.35 Further Reading 34

Part 6 of 13 · Chapter 35 of 64. Next: Chapter 36 — Why MCP Exists (Part 7 opens).

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Read a multi-tool task as a **dependency graph**, and say why its depth, not its node count, sets your round-trip floor.
2. Distinguish **sampling round trips** from **HTTP round trips**, and predict which mechanisms collapse which.
3. Diagnose **under-batching** and **over-batching**, and explain why the documented health metric rewards the second.
4. Account for a long chain's context growth, and choose between prompt caching, context editing, programmatic tool calling and tool search on the evidence of where your tokens are.
5. Configure the **tool search tool** — both variants, `defer_loading`, the response blocks and the continuation rules — and reproduce its documented rejections.
6. Decide whether deferring a catalogue is cheaper than caching it, from a crossover you can compute rather than a rule you were told.
7. Choose between a hand-written loop, the SDK **Tool Runner** and Claude Managed Agents, and say what each one takes away.
8. Assemble the five documented Anthropic tool combinations, and say what each pairing is for.

NOTE

Prerequisites

Chapter 11 for the two-term latency model and the verified rate card. **Chapter 16** for statelessness — the conversation is a list you resend — measured at 26.2× over thirty turns. **Chapter 28** for the seven stop reasons, `pause_turn` as the *server's* loop at a default of ten iterations, and the unresolved `server_tool_use` block. **Chapter 30** for prompt caching from the mechanism outward: writes only at breakpoints, a backward lookback of twenty, and the prefix order `tools` → `system` → `messages`. **Chapter 31** for the client loop, parallel tool use, and the documented health metric. **Chapter 32** for the tool-use system prompt, `defer_loading` and the strict-mode budget. **Chapter 33** for names as retrieval keys under tool search. **Chapter 34** for the error contract, the four-criterion scorer and the leak scanner this chapter reuses.

Deferred by name: **tools whose graphs you did not design** arrive with MCP in **Part 7 (Chapters 36–40)**; **planning and replanning as a first-class activity** is **Chapter 49**; **orchestrator–subagent topologies** are **Chapter 50**; **the context budget in general** is **Chapter 52**, **retrieval and chunking Chapter 53**, and **compaction and memory Chapter 54**; **tracing a chain** is **Chapter 57**.

Introduction: Seven Calls, Four Turns, and a Metric That Lied

An incident-triage assistant does the same seven things every time. It searches the ticket queue, checks service health, pulls the deploy history, opens the top ticket, fetches metrics for the window around the last deploy, correlates all of it, and posts a summary.

Seven tools. The team's first version made seven round trips, one call per turn, because that is what a loop written in an afternoon does. Latency was about eleven seconds and nobody minded.

The second version batched. Someone had read Chapter 31, knew that all results belong in one user message, and knew the documented health metric: `total_tool_calls / tool_call_messages`, which *"should be > 1.0."* They set up a system prompt asking Claude to call everything it could in one go, watched the ratio climb from **1.00 to 2.75**, and shipped it.

The ratio went up. The latency did not go down.

What had happened is that four of the seven calls depend on the output of another call. You cannot open the top ticket before you have searched for tickets. Asking for all seven at once produces three calls that work and four that are guesses — and a guess at a ticket id is a call that must be retried once the real id exists. The chain still took four turns, because four is the depth of the graph and no amount of enthusiasm compresses it. What the second version bought was **four extra tool calls, four extra error results permanently in the conversation, and a metric that now read better than the correct plan's 1.75.**

That is this chapter's subject. Chapters 31 to 34 dealt with a tool call: how it works, how to shape it, how to describe it, how to fail it safely. Chapter 35 closes Part 6 by dealing with *many of them*, in sequence, over time — and the mechanics of one call turn out to be a poor guide to the economics of forty.

Three facts carry the chapter.

A chain is a dependency graph, and its depth is your round-trip floor. Seven nodes, four levels: §35.16 computes both. No prompt, no configuration and no model makes it three.

"Round trip" is two different quantities. Programmatic tool calling reduces *sampling* round trips from seven to one, and reduces HTTP round trips not at all — the documentation is explicit that when a tool function is called, *"code execution pauses and the API returns a* `tool_use` *block"*. What it does eliminate is the intermediate results: *"Tool results from programmatic invocations do not count toward your input/output token usage."* Which of those two numbers you care about depends on whether your problem is latency or context.

A chain's cost is dominated by history, not by tools. §35.17 accounts a thirty-turn chain and finds **65.6 per cent of raw input tokens are carried history** rather than prefix. Four ways of attacking that are documented, they target four different sources, and the chapter measures all four rather than recommending one.

There is a fourth thing, and it is the one most likely to save you money this week. Everyone knows tool search cuts context by 85 per cent for a big catalogue. §35.18 computes the crossover and finds that below roughly **35 tools, under prefix-only caching, deferring is more expensive than caching** — at ten tools it costs **3.5 times more**. The documentation says as much in words; the chapter says it in a number you can check against your own catalogue.

35.1 Core Concepts

> DEFINITION
>
> **Chain**
>
> A sequence of tool calls in which at least one call's input depends on another call's output. A chain is not a list; it is a directed acyclic graph, and treating it as a list is the origin of both failure modes in this chapter.

DEFINITION

Level

A set of nodes in a dependency graph whose dependencies are all already satisfied. Every node in a level may be called in the same turn, and that is exactly what parallel tool use is for.

DEFINITION

Depth

The number of levels — the critical path. Depth is the minimum number of assistant turns a chain requires. Node count is not.

DEFINITION

Sampling round trip

One generation: the model is run and produces content. Latency here is Chapter 11's two-term model, dominated by decode.

DEFINITION

HTTP round trip

One `POST /v1/messages`. Usually the same thing as a sampling round trip, but not under programmatic tool calling, where a request may return a paused container rather than a fresh generation.

DEFINITION

Under-batching

Splitting independent calls across turns. Legal, invisible, and documented as teaching the model to stop batching.

DEFINITION

Over-batching

Putting a dependent call in the same turn as the call that produces its input. The model must invent the input; you pay for the invention and for the recovery.

DEFINITION

Deferred tool

A tool carrying `defer_loading: true`. Its full definition is still sent in the `tools` array on every request, but the API keeps it out of the system-prompt prefix until Claude discovers it through tool search.

DEFINITION

`tool_reference`

The block a search returns naming a discovered tool. The API expands it into the full definition before Claude sees it; you never expand one yourself.

DEFINITION

Orchestration

Deciding what runs, in what order, in which turn, and where. In Part 6 the orchestrator is your loop. In Chapters 49 to 51 it becomes a component with a plan of its own.

35.2 A Chain Is a Graph

Take the triage chain from the introduction and write down only the dependencies:

Call	Depends on
`search_tickets`	—
`get_service_health`	—
`get_deploy_history`	—
`get_ticket_detail`	`search_tickets`
`get_metrics`	`get_deploy_history`
`correlate`	`get_ticket_detail`, `get_metrics`, `get_service_health`
`post_summary`	`correlate`

`chainplan.levels` groups these into the sets that may run together:

```
level 0  get_deploy_history, get_service_health, search_tickets
level 1  get_metrics, get_ticket_detail
level 2  correlate
level 3  post_summary
```

Seven nodes, **depth four**. That four is not a target or a heuristic; it is a property of the problem, and it is the smallest number of assistant turns the chain can take with client tools. If four turns is too slow, the fix is to change the graph — merge `correlate` into `post_summary`, or consolidate `get_ticket_detail` behind the search — which is Chapter 32's consolidation argument arriving from a different direction.

KEY TAKEAWAYS

The rule

Depth is the floor; node count is the ceiling. Every strategy in this chapter sits between the two, and the only way below the floor is a different graph.

35.3 Four Ways to Execute the Same Graph

§35.16 runs all four over the same seven nodes.

Strategy	Sampling	HTTP	Tool calls	Wasted	Health ratio	Results in context
One per turn	7	7	7	0	1.00	yes
Level-batched	4	4	7	0	1.75	yes
All at once	4	4	11	4	2.75	yes
Programmatic	1	5	7	0	1.75	**no**

One per turn is the accidental default. Chapter 31 quoted the consequence of returning each result in its own user message — it *"'teaches' Claude to avoid parallel calls"* — and here that habit costs **+75 per cent sampling round trips** over the correct plan, on a graph that has not changed.

Level-batched is correct. Each turn asks for exactly the calls whose inputs exist. This is what Chapter 31's documented system-prompt line is for: *"Only batch tool calls that are independent of each other."*

All at once is the introduction's mistake, and its numbers are the most interesting in the table. It does **not** cost round trips: level 0 succeeds in the optimistic turn and the remaining three levels recover normally, so depth is still four. What it costs is **four wasted calls**, four `is_error` results that sit in the conversation for the rest of its life, and — on a tool with a side effect — Chapter 34's whole subject, because a guessed input to a write is a wrong write.

Programmatic is the one that changes the shape rather than the schedule. Claude is sampled **once**; the script runs in the sandbox and pauses on each tool call, so the HTTP round trips do not collapse. The win is that the intermediate results never enter the conversation, which §35.17 prices at **8.09×**.

> WARNING
>
> **The metric ranks the wrong plan first**
>
> The documented health signal is `total_tool_calls / tool_call_messages`, *"should be > 1.0"*. Over-batching scores **2.75**. The correct plan scores **1.75**. A dashboard that treats the ratio as a target rather than a floor will reward the plan that guesses inputs. Use it to detect under-batching, which is what it was offered for, and use the wasted-call count to detect the other direction.

35.4 What a Chain Does to the Context Window

Chapter 16 measured statelessness on a chat: thirty turns, 54,158 tokens against a naive 2,070, a factor of 26.2. A chain is worse-shaped than a chat, because every turn adds two blocks rather than one, and the `tool_result` block is the big one.

§35.17 accounts a thirty-turn chain with a 6,400-token tools array, a 1,200-token system prompt, and 850-token results. Raw input across the run: **663,000 tokens, of which 65.6 per cent is carried history**. Against a naive model that charges for one turn thirty times, the ratio is **2.57×** — lower than Chapter 16's 26.2 precisely because the prefix here is large, which is a reminder that the statelessness

multiplier is a property of your token distribution rather than a constant.

Four documented approaches attack four different sources, and the `manage-tool-context` page sets them out plainly (verified 4 August 2026):

Approach	What it reduces	When it fits
Tool search	Tool definitions loaded upfront	Large toolsets (20+) where most tools aren't needed every turn
Programmatic tool calling	`tool_result` round trips	Chains that can execute as a single script
Prompt caching	The *cost* of repeated definitions	Stable toolsets across many requests
Context editing	Old `tool_result` blocks in history	Long conversations where early results are stale

The documented starting order is worth quoting because it is a sequence, not a menu: caching **from day one**; tool search once the toolset passes roughly twenty tools or baseline context becomes noticeable; context editing once conversations run long enough that early results are irrelevant; programmatic tool calling if you notice *"repetitive chains of small tool calls that could run as a single batch."*

§35.17 prices the last three against a prefix-cached baseline on the same chain: a rolling message cache **4.31×**, context editing to the last five results **2.21×**, and keeping results out of context entirely **8.09×**. They compose — the page says so explicitly — and none of them is a substitute for having the right graph.

35.5 Tool Search: Discovery as Retrieval

The tool search tool is how a catalogue stops being a prefix and starts being an index. The mechanics, verified 4 August 2026:

1. You include a tool search tool in `tools`.
2. You send **every** definition in `tools`, with `defer_loading: true` on the ones that should not load upfront. At least one tool — normally the search tool — must stay non-deferred.
3. Claude's context initially holds only the search tool and the non-deferred set.
4. Claude searches; the API returns matching tools as `tool_reference` blocks, **up to 5 by default**, and expands them into full definitions.

Two variants, and the difference is not cosmetic:

Variant	Query kind	Limit
`tool_search_tool_regex_20251119`	Python `re.search()` patterns, case-insensitive	200 characters
`tool_search_tool_bm25_20251119`	Natural language	500 characters

Both search **names, descriptions, argument names and argument descriptions** — which is Chapter 33's argument made operational: the description you wrote for selection is now also the index you are retrieved from. Undated aliases `tool_search_tool_regex` and `tool_search_tool_bm25` resolve to the latest dated version.

The documented numbers that motivate the feature are worth memorising. A typical five-server setup — GitHub, Slack, Sentry, Grafana, Splunk — *"can consume ~55k tokens in definitions before Claude does any work"*, and tool search *"typically reduces this by over 85 percent, loading only the 3–5 tools Claude needs for a given request."* Separately, and more importantly: *"Claude's ability to pick the right tool degrades once you exceed 30–50 available tools."*

Limits: **10,000 deferred tools** per request, 5 results per search by default, and the model list covers Fable 5, Mythos 5, Opus 5, Opus 4.8 through 4.5, Sonnet 4.6, Sonnet 4.5 and **Haiku 4.5** — Opus 4.1 and earlier do not support it.

The caching interaction, which is the part people get wrong

Deferred tools are excluded from the rendered tools section **before the cache key is computed**, so adding them cannot invalidate a prefix. That is the headline and it is true. The sentence next to it is the one that costs money: when a deferred tool is discovered, *"the tool's full definition is expanded inline at that point in the conversation body, not in the prefix."*

The conversation body is the part Chapter 30 showed is hardest to cache and is resent every turn. So a discovered definition moves from a 0.1× line to a 1.0× line — unless your breakpoints cache the message history too. $35.18 turns that into a crossover.

> COMMON MISTAKE
>
> **Deferring a small catalogue**
>
> **Symptom.** Tool search is enabled, the context window looks tidier, and the bill goes up.
>
> **Cause.** Under prefix-only caching a cached definition costs 0.1× per turn forever, while a discovered one sits in the body at 1.0×. With ten tools at 460 tokens each, the whole catalogue cached costs **460 units a turn** and three discovered tools cost **1,626 — 3.5× more**, for a catalogue small enough that selection was never the problem.
>
> **Fix.** Compute the crossover for your own token sizes. $35.18 puts it at about **35 tools (16,260 definition tokens)** with prefix-only caching, and about **8 tools** if your breakpoints cache the message history.
>
> **Detection.** Compare `cache_read_input_tokens` against `input_tokens` before and after enabling deferral. If `input_tokens` rose more than `cache_read_input_tokens` fell, you have moved definitions out of the cheap column.

The honest framing is that tool search is primarily an **accuracy and scale** feature that usually also saves money, rather than a cost feature. Above 30 to 50 tools the documented reason to use it is that selection degrades, and no caching arrangement fixes that.

35.6 What You Send Back After a Search

The response introduces two block types your reader must know, and one rule that catches everyone.

```
{ "type": "server_tool_use", "id": "srvtoolu_01ABC123",
  "name": "tool_search_tool_regex", "input": { "pattern": "weather" } }
{ "type": "tool_search_tool_result", "tool_use_id": "srvtoolu_01ABC123",
  "content": { "type": "tool_search_tool_search_result",
               "tool_references": [
                 { "type": "tool_reference", "tool_name": "get_weather" }
               ] } }
```

- The search runs on Anthropic's servers. **Never return a `tool_result` for the `srvtoolu_...` id** — the API rejects the request.
- Pass the assistant content back **unchanged**, including both blocks, and send the same `tools` array: the search tool plus every deferred definition.
- The API expands `tool_reference` blocks throughout the history, so a discovered tool can be reused in later turns without re-searching.
- A search that matches nothing returns an **empty `tool_references` array, not an error**.

Two 400s and four result-level error codes are documented, and §35.18 reproduces all of them offline. The 400s are *"At least one tool must have defer_loading=false"* and *"Tool reference '...' not found in available tools"*. The result-level codes, arriving inside a 200 as `tool_search_tool_result_error`, are `invalid_tool_input`, `unavailable`, `too_many_requests` and `execution_time_exceeded`.

One more: a tool with `defer_loading: true` **cannot carry** `cache_control` — the API returns a 400. Put the breakpoint on a non-deferred tool.

> TIP
>
> **Strict mode and deferral compose**
>
> The grammar for strict mode builds from the **full** toolset, so `defer_loading` and `strict: true` work together with no grammar recompilation. Chapter 23's grammar cache and Chapter 30's prompt cache remain two different caches with two different keys; deferral touches one of them.

35.7 You Can Write the Search Yourself

If regex and BM25 do not suit your catalogue — you have embeddings, or a taxonomy, or a permission model that should filter what is discoverable — the documented escape hatch is a **custom client-side tool search**. Define an ordinary tool, and return `tool_reference` blocks in a standard `tool_result`:

```
{ "type": "tool_result", "tool_use_id": "toolu_your_tool_id",
  "content": [ { "type": "tool_reference",
                 "tool_name": "discovered_tool_name" } ] }
```

Every referenced tool still needs a definition in the top-level `tools` array, normally with `defer_loading: true`, and the API expands the references the same way. Note the shape difference: the server-side format is `tool_search_tool_result`; a client-side implementation uses the ordinary `tool_result` envelope. Getting that backwards is the common error, and it is why the documentation says so explicitly.

This is the seam where Part 6 meets Chapter 53. A custom tool search is a retrieval system whose corpus is your tool descriptions, and everything Chapter 53 says about chunking, embeddings and recall applies

to it.

35.8 Who Runs the Loop

There are three documented answers, and they trade control for code in a straight line.

Your own loop. Chapters 31 and 34 built it. You own the continuation rules, the gates, the idempotency and the error strings. The documentation's guidance on when this is right is specific: use the manual loop *"when you need human-in-the-loop approval, custom logging, or conditional execution."*

The SDK Tool Runner. In beta, available in all seven SDKs. It runs tools when Claude calls them, handles the request/response cycle, manages conversation state, and provides type safety and validation. It *"loops until Claude returns a message without a tool use, or until it reaches* `max_iterations` *if you set it"* — and all seven SDKs support `max_iterations`. Python, TypeScript and Ruby additionally offered client-side compaction, now **deprecated in favour of server-side context editing**; Go, Java, C# and PHP never had it.

Claude Managed Agents. Chapter 16 named this as the alternative to building the loop at all. It is a Part 9 subject and Chapter 47 takes it up.

The Tool Runner is a good default, and it has one behaviour this chapter has to flag because Chapter 34 already measured it. The documentation is precise: *"When a tool throws an exception, the tool runner catches it and returns the error to Claude as a tool result with* `is_error: true`*. The tool result carries the exception's message (in Python, its type and message), not the full stack trace."*

Score that string with Chapter 34's scorer and $35.19 gets **0 out of 40 actionable and 36 out of 40 leaking** — identical to Chapter 34's raw-exception arm on both axes. Removing the traceback removes the two disclosure patterns that live in a traceback (`stack_trace`, `stack_frame`) and none of the four that live in the message (`abs_path`, `conn_string`, `internal_host`, `private_ip`), and it removes none of the four things a retry needs, because none of them was ever in a frame.

That is not a criticism of the default. A framework cannot know what your tool's failures mean. It is an instruction: **if you use the Tool Runner, catch your own exceptions inside the tool and return Chapter 34's instructive string yourself**, so the runner never sees an exception it has to translate.

35.9 The Documented Tool Combinations

Anthropic publishes five pairings, described as *"starting points, not prescriptions"* (verified 4 August 2026). Each is a chain shape rather than a feature list.

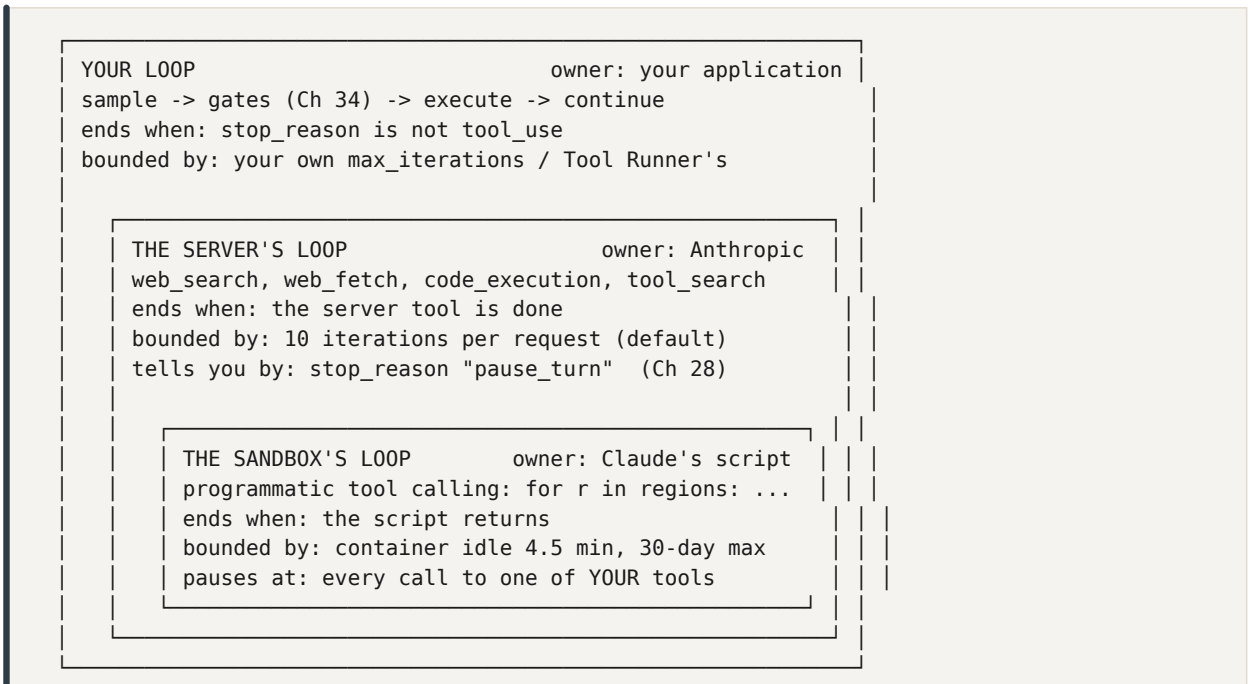
Pattern	Tools	The shape
Research agent	`web_search` + `code_execution`	Search finds sources; code analyses and synthesises. Search, execute, optionally search again for a gap.
Coding agent	`text_editor` + `bash`	Inspect, edit, run tests, repeat. Both client-executed, so your application controls files and commands.
Cite-then-fetch	`web_search` + `web_fetch`	Search surfaces candidates; fetch retrieves only the two or three that look relevant.

Pattern	Tools	The shape
Long-running agent	`memory` + any toolset	Memory is orthogonal: it persists state without changing how other tools behave.
All-in-one	`computer`	Subsumes the rest by driving a desktop; *"the most general option and also the slowest, since every action requires a screenshot roundtrip."*

Two of these are worth a sentence each in the language of this chapter. **Cite-then-fetch is a two-level graph with a filter between the levels**, and the filter is the point — fetching everything the search returned is over-batching with a bandwidth bill. **The coding agent is a cycle, not a chain**: inspect, edit, test, and back to inspect. A cycle has no depth, which is why `max_iterations` exists and why Chapter 49 is about replanning rather than planning.

35.10 Where the Loops Are

Three loops run in a tool-using system and they are owned by three different parties. Confusing them is the most common source of a "why did it stop?" question.



Only the outermost loop executes YOUR client tools. The middle one never asks you for anything. The innermost one asks you for a tool_result and then carries on without re-sampling Claude.

The innermost loop is the one that repays study, because it is the only place in the API where **your tool runs without the model being consulted first**. That is exactly why the documentation's warning about container expiry matters: *"you must respond before the container expires... If the container expires, Claude may treat the tool call as timed out and retry it."* Chapter 34's word for a retry that may or may not have executed is *duplicate*.

35.11 Analogies, and Where They Break

VISUAL ANALOGY

A kitchen ticket rail

A chef reads a ticket and starts everything that can start now: the stock goes on, the oven preheats, the vegetables are prepped. What cannot start is the plating, because there is nothing to plate. The rail is a dependency graph and the chef is a level scheduler, and a good one is fast not because they cook faster but because they never wait for something they could have started earlier.

Where it breaks. The chef holds the whole graph in their head between orders; your model holds only what is in the context window, and Chapter 16's statelessness means every level's history is re-read from scratch. And a chef who starts the plating early has wasted a plate; a model that calls a write tool early has, per Chapter 34, moved money.

VISUAL ANALOGY

A library with a card catalogue

Before tool search, every tool definition was a book piled on the reading desk: instantly available, and after fifty books you cannot see the desk. Tool search is the card catalogue — one small index, and you fetch the three books you need.

Where it breaks. The catalogue only pays once the shelves are big. With five books the desk was fine, and walking to the catalogue costs a trip you did not need — which is $35.18's crossover, and which the library analogy gets right for reasons of physical space rather than caching. The deeper mismatch: a fetched book goes back on the shelf, whereas a discovered tool definition stays in the conversation body and is re-read on every subsequent turn.

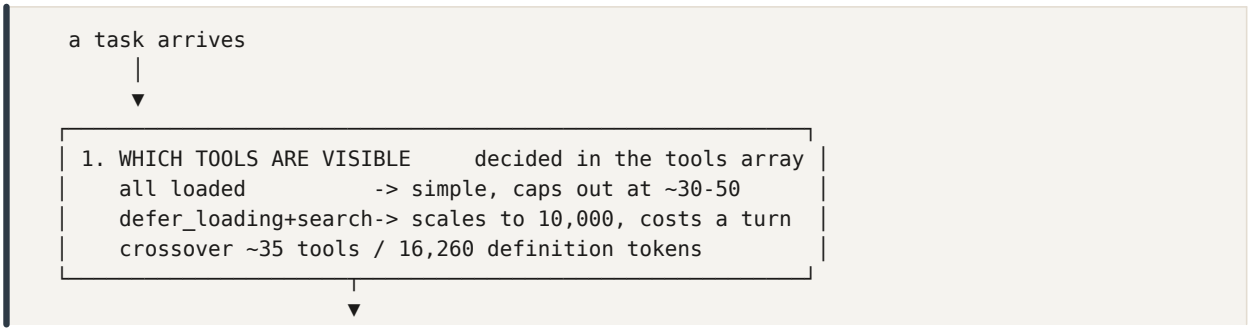
VISUAL ANALOGY

A courier who waits at the door

Programmatic tool calling is a courier who takes your whole shopping list, goes to the shops, and comes back once — but knocks each time they need your card. The trips to the door still happen. What you are spared is explaining the list again at every knock.

Where it breaks. A courier remembers the list; the container forgets after 4.5 minutes idle. And a real courier will improvise if a shop is shut, whereas the script does what it was written to do at sampling time, which is why the documentation notes that a timeout in the sandbox surfaces in `stderr` and prompts a retry rather than a rethink.

35.12 Architecture: Where the Decisions Live



```
┌──────────────────────────────────────────────────────────┐
│ 2. WHAT SHAPE IS THE TASK      decided by the graph      │
│    depth = round-trip floor; level = one fan-out         │
│    batch a LEVEL, never a chain                          │
└──────────────────────┬───────────────────────────────────┘
                       ▼
┌──────────────────────────────────────────────────────────┐
│ 3. WHERE DOES THE CHAIN RUN    decided by the tool list  │
│    your loop     -> full control, results in context     │
│    server tools  -> pause_turn at 10, you never execute  │
│    code execution-> 1 sampling trip, results OUT of ctx  │
└──────────────────────┬───────────────────────────────────┘
                       ▼
┌──────────────────────────────────────────────────────────┐
│ 4. WHAT DOES HISTORY COST      decided by breakpoints    │
│    prefix cached only          466,540 units / 30 turns  │
│    + rolling message cache     108,140  (4.31x cheaper)  │
│    + context editing, last 5   211,540  (2.21x cheaper)  │
│    results out of context       57,640  (8.09x cheaper)  │
└──────────────────────┬───────────────────────────────────┘
                       ▼
            the gates of Chapter 34, per call
```

Decisions 1, 3 and 4 are made before the conversation starts and are hard to change during it: the `tools` array is the first cache prefix level, and changing it invalidates everything (Chapter 30). Decision 2 is made turn by turn, by the model, and is the one you influence with a sentence in the system prompt. That asymmetry is worth internalising — **most of your orchestration is configuration, and only the batching is prompting.**

35.13 Worked Example: Reading a Chain Off a Transcript

You inherit an agent and want to know whether its orchestration is sound. You do not need a profiler; you need four counts from the message history.

Count one: assistant messages containing at least one `tool_use` block. That is `tool_call_messages`. Call it *M*.

Count two: `tool_use` blocks in total. That is `total_tool_calls`, *C*. The documented ratio is *C/M*, and it *"should be > 1.0"*.

Count three: `tool_result` blocks with `is_error: true` whose content says the input was missing or wrong. Call it *W*, for wasted. This is the count the documented metric does not have, and it is what separates the correct plan from the enthusiastic one.

Count four: distinct levels. Reconstruct the graph by matching each call's input values against earlier results. *M* should equal the depth.

Now read the four together:

- *C/M* = 1.00 → under-batching. The model is calling one tool per turn. Check whether your history returns results one message at a time.
- *C/M* > 1 and *W* = 0 → level-batched. This is what you want.
- *C/M* high and *W* > 0 → over-batching. The ratio is being inflated by calls that failed for want of an input. Add the documented line — *"Only batch tool calls that are independent of each other"* — to the

system prompt and re-measure.

- M > depth → something is re-planning between levels. That is not necessarily wrong, but it is Chapter 49's subject rather than this one's.

For the triage chain, the four plans give (M, C, W): one-per-turn (7, 7, 0), level-batched (4, 7, 0), all-at-once (4, 11, 4), programmatic (1, 7, 0). Only the third has a non-zero W, and only the fourth has M below depth — because in the fourth, three of the levels happened inside a sandbox and were never sampled at all.

35.14 Code Walkthrough: Levels, and Why the Planner Is Six Lines

The whole of this chapter's scheduling argument fits in one function, and its shortness is the argument.

```
def levels(graph):
    """Group nodes into dependency levels. Level i may run in one turn."""
    remaining = dict(graph)
    done, out = set(), []
    while remaining:
        ready = sorted(n for n, deps in remaining.items()
                       if all(d in done for d in deps))
        if not ready:
            raise CycleError("cycle among %s" % sorted(remaining))
        out.append(ready)
        done.update(ready)
        for n in ready:
            del remaining[n]
    return out
```

`ready` is the definition of "may be batched": every dependency already satisfied. Sorting makes the output deterministic, which matters because a test asserts on it and because a non-deterministic plan is not a plan. The `if not ready` branch is the only error path — a graph with a cycle has no level structure, and raising is better than looping, which is the same instinct Chapter 28's reader applied to an unknown stop reason.

Two things this function deliberately does not do. It does not weight nodes by latency, so a level containing one slow tool and four fast ones is scheduled as one turn — correct, because the turn ends when the slowest result arrives regardless. And it does not merge levels, because merging levels is over-batching and the whole point is to stop.

The three planners are then arithmetic over the levels:

```
def all_at_once(graph):
    lv = levels(graph)
    wasted = len(graph) - len(lv[0])
    rounds = 1 + (len(lv) - 1)
    return Plan("all_at_once", rounds, rounds, len(graph) + wasted,
                wasted, rounds)
```

`wasted` is every node outside level 0: those are the calls whose inputs do not exist yet, so they are guesses. `rounds` is one optimistic turn plus the remaining levels, which is why over-batching costs no

depth on this graph. The function returns a `Plan` whose `batching_ratio` property computes the documented health metric — and computing it here, next to `wasted`, is what makes $35.16's negative result visible rather than theoretical.

The second walkthrough is the crossover, which is four lines and one honest parameter.

```
def steady_state_units(n_tools, tokens_per_tool=460, hot=4,
                       discovered=3, search_tool=620,
                       cache_messages=False):
    total = n_tools * tokens_per_tool
    baseline = 0.1 * total
    prefix = min(hot, n_tools) * tokens_per_tool + search_tool
    body_rate = 0.1 if cache_messages else 1.0
    deferred = 0.1 * prefix + body_rate * discovered * tokens_per_tool
    ...
```

`baseline` is the whole catalogue in the cached prefix, read at 0.1× every turn. `deferred` is the search tool plus the hot set in the prefix at 0.1×, **plus the discovered definitions in the conversation body**, whose rate is the only argument in the function. Set `cache_messages=True` and the body rate drops to 0.1×, which moves the crossover from about 35 tools to about 8. The parameter is exposed rather than assumed because it is a decision you make with your breakpoints, and a function that hid it would be asserting your cache configuration on your behalf.

35.15 Mini Project: A Chain, Priced Four Ways

What it is. A planner, a context accountant, a tool-search economics model with its documented rejections, and a scoring pass over the SDK Tool Runner's default error string. Everything runs offline with no key, extends the code from Chapters 11, 30, 31, 32 and 34, and publishes two negative results.

```
ch35/
├── reader.py       ─┐ copied from ch34/ (via ch31/, ch28/)
├── agentloop.py     │ the continuation gate
├── errmsg.py        │ the four-criterion scorer and leak scanner
├── errcorpus.py     │ 40 BUILT failures x 5 arms
├── callsite.py      │ the validator derived from ch32's audit
├── enforce.py       │
├── schema.py        │
├── tooldef.py       │
├── catalogue.py    ─┘
├── evalkit.py         Wilson intervals (ch33/, via ch8/)
├── costmodel.py       the verified rate card (ch11/)
├── cacheplan.py       breakpoints and multipliers (ch30/)
│
├── chainplan.py       NEW  levels, depth, four execution strategies
├── ctxgrow.py         NEW  30-turn context accountant, four regimes
├── toolsearch.py      NEW  defer_loading economics + 6 rejections
├── runnerarm.py       NEW  the Tool Runner's default, scored
├── report.py          NEW  every published figure
└── test_ch35.py       NEW  28 tests
```

```
cd ch35
pip install --break-system-packages pytest
python3 -m pytest -q      # 28 passed
```

```
python3 report.py
```

Provenance. The tool search limits, variants, error codes, rejections and the ~55k and 30–50 figures are **DOC**, transcribed on 4 August 2026. The triage graph, the block sizes in `ctxgrow.py` and the 460-tokens-per-tool figure are **BUILT** parameters, stated as such — and the last of them is checkable: 120 tools at 460 tokens is 55,200, against the documented *"~55k"* for a five-server setup.

35.16 Experiment One: Four Ways to Run Seven Calls

```
1. A chain is a graph; its depth is your round-trip floor
----------------------------------------------------------------------
nodes 7, depth 4
  level 0  get_deploy_history, get_service_health, search_tickets
  level 1  get_metrics, get_ticket_detail
  level 2  correlate
  level 3  post_summary
  strategy       sampling   http  calls  wasted    ratio results in ctx
  one_per_turn          7      7      7       0     1.00 True
  level_batched         4      4      7       0     1.75 True
  all_at_once           4      4     11       4     2.75 True
  programmatic          1      5      7       0     1.75 False
  under-batching costs +75% sampling round trips;
  over-batching costs +57% tool calls at the same depth.
```

Read the columns rather than the rows. **Sampling** is what Chapter 11's latency model charges for; **HTTP** is what your timeout budget and Chapter 29's retry policy see; **calls** is what your tools' dependencies see; **wasted** is what Chapter 34's gates see.

Programmatic tool calling is the row that separates the first two columns: one sampling round trip, five HTTP round trips. The documentation is unambiguous about why — when a tool function is called, *"code execution pauses and the API returns a* `tool_use` *block"* — and equally unambiguous about the compensation: *"Tool results from programmatic invocations do not count toward your input/output token usage."* If your problem is latency from re-sampling, this collapses it. If your problem is the number of network exchanges, it does not.

> WARNING
>
> **The negative result about the documented health metric**
>
> `total_tool_calls / tool_call_messages` is documented as *"should be > 1.0"*. On this graph, over-batching scores **2.75** and the correct plan scores **1.75**. The metric ranks the wrong plan first.
>
> It is not a bad metric; it is a **floor**. It detects the failure it was offered for — under-batching, which scores exactly 1.00 — and it is blind to the opposite failure by construction, because a wasted call is still a call. Pair it with the wasted-call count or it will reward guessing.

35.17 Experiment Two: A Thirty-Turn Chain

A 6,400-token tools array, a 1,200-token system prompt, 850-token results, thirty turns, one call per turn, priced on Opus 5.

```
2. What a thirty-turn chain does to the bill
------------------------------------------------------------------
  regime                                  raw in  carried    billed  dollars
  prefix cached only                      663000    65.6%    466540   2.3327
  prefix + rolling message cache          663000    65.6%    108140   0.5407
  context editing, last 5 results         408000    44.1%    211540   1.0577
  programmatic (results out of context)   254100    10.3%     57640   0.2882
  a rolling message cache cuts the bill 4.31x
  context editing to the last 5 results cuts it 2.21x
  keeping results out of context entirely cuts it 8.09x
  statelessness (Ch 16) recomputed for a chain: 663000 tokens
  against a naive 258000, a factor of 2.57x
```

Four observations, in the order they change what you would do.

Two-thirds of the input is history. 65.6 per cent of the raw input tokens across thirty turns are carried `tool_use` and `tool_result` blocks rather than the prefix. Any optimisation aimed at the tools array is aimed at the smaller third.

A rolling message cache is the cheapest intervention and requires no design. It is a breakpoint placement, and it beats context editing here by nearly 2×. That ordering is not universal — it depends on how much of the history is stable — but it is the reason the documented starting order puts caching first and context editing third.

Context editing changes the raw token count; caching changes only the bill. The third row is the only one whose `raw in` falls, from 663,000 to 408,000. If your problem is the context *window* rather than the invoice, caching does nothing for you and editing does.

Chapter 16's 26.2× is not a constant. The same phenomenon on this workload is 2.57×, because a chain carries a big fixed prefix that a chat does not. The multiplier is a property of your token distribution. Quote the mechanism, never the number.

35.18 Experiment Three: What Deferring Actually Costs

```
3. Discovery is retrieval, and deferring is not free
------------------------------------------------------------------
   tools def tokens    baseline    deferred cheaper?
       5       2300         230        1626 BASELINE
      10       4600         460        1626 BASELINE
      20       9200         920        1626 BASELINE
      35      16100        1610        1626 BASELINE
      40      18400        1840        1626 deferred
     120      55200        5520        1626 deferred
  crossover, prefix cached only: 16260 tokens (~35 tools)
  crossover, messages cached too: 3840 tokens (~8 tools)
  the documented 5-server setup: 55200 definition tokens,
  5520 units per turn cached against 1626 deferred (71% less)
```

At the documented five-server scale, deferring is **71 per cent cheaper per steady-state turn** — in the same territory as the published *"over 85 percent"* reduction in context, which is a different quantity measured a different way and should not be expected to match exactly.

Below the crossover the sign flips. At ten tools the whole catalogue cached costs **460 units a turn** and three discovered definitions cost **1,626** — **3.5× more** — because a cached definition is billed at 0.1× and a definition expanded into the conversation body is billed at 1.0× until a breakpoint reaches it.

The crossover lands at about **35 tools** with prefix-only caching and about **8 tools** with a rolling message cache. Note where 35 sits: the documentation says selection accuracy *"degrades once you exceed 30–50 available tools."* **The token crossover and the accuracy threshold land in the same region**, which is a pleasant coincidence and not a coincidence at all — both are consequences of definitions being large — but only one of them is about money, and the accuracy one is the reason that should decide it.

The documented rejections, all reproduced with no key:

```
  documented rejections, reproduced offline:
    every tool deferred                          rejected
    cache_control on a deferred tool             rejected
    tool_reference naming an absent tool         rejected
    regex pattern over 200 characters            rejected
    bm25 query over 500 characters               rejected
    tool_result returned for the search's srvtoolu_ id  rejected
  6 of 6 reproduced with no key
```

The last one is the one that bites in practice, because a `server_tool_use` block looks exactly like a `tool_use` block to a loop that switches on block presence rather than block type. Chapter 28's reader already distinguishes them; `toolsearch.check_continuation` refuses at the point where the request is assembled.

35.19 Experiment Four: The Tool Runner's Default, Scored

Chapter 34 scored five ways of writing an `is_error` string. This is the sixth, and it is the one most systems will actually ship, because it is what the SDK does when your tool raises.

```
4. The tool runner's default error string, scored (PROXY)
------------------------------------------------------------------
  arm                    field  value  constraint  next_step  actionable leaks
  terse                      0      0           0          0       0/40   0/40
  raw                       16     16          24          0       0/40  36/40
  redacted_raw              16     16          24          0       0/40   0/40
  instructive               40     40          40         40      40/40   0/40
  instructive_verbatim      40     40          40         40      40/40  36/40
  tool_runner               16     16          24          0       0/40  36/40
  disclosure kinds, raw       : abs_path 22, conn_string 11,
                                internal_host 6, private_ip 11,
                                stack_frame 3, stack_trace 3
  disclosure kinds, tool_runner: abs_path 19, conn_string 11,
                                internal_host 6, private_ip 11
```

The documented behaviour is that the runner *"carries the exception's message (in Python, its type and message), not the full stack trace."* Removing the stack trace removes exactly the two disclosure patterns that live in a stack trace, and leaves `abs_path`, `conn_string`, `internal_host` and `private_ip` untouched, because those live in the *message* — which is where a well-written exception puts the thing that broke.

Actionability is unchanged at **0 out of 40**, and the reason is structural: none of the four things a retry needs was ever in a frame. `next_step` scores zero for both arms because an exception is a diagnosis and an instruction is a different genre.

The conclusion is an instruction rather than a complaint. **If you use the Tool Runner, catch the exception inside your tool and return Chapter 34's instructive string yourself.** The runner is then never asked to translate something it cannot understand, and the only arm in the table that is both actionable and clean is the one you wrote.

TIP

Where the runner still helps

`max_iterations` — supported by all seven SDKs — is the bound on your outer loop that Chapter 29's `max_retries` structurally cannot provide, in the same way that a fleet-wide budget is not `max_retries`. A cyclical toolset like the documented `text_editor` + `bash` coding agent has no depth to bound it, and `max_iterations` is what stops a loop that has decided to keep going.

35.20 Hands-on Labs

INDUSTRY USE CASE

Lab 1 — Draw your graph

Take a real multi-tool task from your system and write down, for each call, which earlier call produced each of its inputs. Run `chainplan.levels` over the result.

Success criterion. You can state the depth, and you can name at least one pair of calls currently in different turns that belong in the same one.

INDUSTRY USE CASE

Lab 2 — Count the four numbers

From a week of transcripts, compute M, C, W and depth as described in §35.13.

Success criterion. You can classify your agent as under-batched, correct or over-batched, with a wasted-call count to back it up.

INDUSTRY USE CASE

Lab 3 — Find your own crossover

Measure your tool definitions' real token count with `count_tokens` (free, and on an independent rate limit — Chapter 19). Feed the average into `toolsearch.crossover` for both caching regimes.

Success criterion. Two numbers, and a statement of which regime your breakpoints actually implement.

INDUSTRY USE CASE

Lab 4 — Price your history

Instrument one long conversation and record `input_tokens`, `cache_read_input_tokens` and `cache_creation_input_tokens` per turn. Plot the carried share.

Success criterion. You can say what fraction of your input bill is history, and whether caching or editing is the right lever.

INDUSTRY USE CASE

Lab 5 — Replace the runner's default

Pick one tool that currently lets exceptions escape to the Tool Runner. Wrap it so that every failure returns an instructive string, and score both versions with `errmsg`.

Success criterion. The new version scores 4/4 on the criteria and 0 on the leak scanner, for every failure mode you can produce.

35.21 Lab Solutions

Lab 1. The work is the writing-down, not the code. Two rules make it tractable: a dependency exists only if a *value* flows, not if a call merely happens earlier; and if you cannot say which field of which earlier result produced an input, the dependency is in the prompt rather than in the data, and that is a different fix.

```
import chainplan as CP
G = {"list_repos": [], "get_user": [],
     "list_prs": ["list_repos"], "review": ["list_prs", "get_user"]}
print(CP.depth(G), CP.levels(G))
```

Lab 2. Four counts over a transcript, and the third is the one nobody has:

```
M = sum(1 for m in msgs if m["role"] == "assistant"
        and any(b["type"] == "tool_use" for b in m["content"]))
C = sum(1 for m in msgs for b in m["content"]
        if b.get("type") == "tool_use")
W = sum(1 for m in msgs for b in m["content"]
        if b.get("type") == "tool_result" and b.get("is_error"))
```

W over-counts, because it includes genuine tool failures as well as guessed inputs. Split it by whether the error string mentions a missing or invalid parameter — which you can do reliably only if your error strings say so, which is Chapter 34's argument arriving as a debugging convenience.

Lab 3. `count_tokens` is free and rate-limited independently, so measure rather than estimate:

```
n = client.messages.count_tokens(model=MODEL, tools=TOOLS,
                                 messages=[{"role": "user",
                                            "content": "x"}])
per_tool = n.input_tokens / len(TOOLS)
print(toolsearch.crossover(tokens_per_tool=per_tool))
```

If your measured `per_tool` is far above 460, your crossover is *lower* in tool count and you should defer sooner. If it is far below, your descriptions may be too short to be discoverable — Chapter 33's linter has an opinion about that, and its OVERLAP rule is published as blind to exactly this case.

Lab 4. The carried share is derivable from `usage` without any instrumentation of your own:

```
carried = u["input_tokens"] + u["cache_read_input_tokens"] - PREFIX
share = carried / (u["input_tokens"] + u["cache_read_input_tokens"])
```

Watch the share over the conversation rather than at the end. A share that rises past about two-thirds is the signal that $35.17's regimes are worth their configuration.

Lab 5. The wrapper is short and belongs in the tool, not in the runner:

```
def payments_transfer(**kw):
    try:
        return _transfer(**kw)
    except LedgerError as e:
        raise ToolFailure(errmsg.runtime_instructive(
            {"plain": _plain(e), "retryable": _is_transient(e)}))
```

`_plain` maps your exception taxonomy onto sentences you control. That mapping is the deliverable; the `try` is trivial. If you cannot write `_plain` for a failure, you have found a failure whose meaning nobody in your team has decided, and the model was never going to guess it either.

35.22 Best Practices

BEST PRACTICE

The fourteen rules

1. **Draw the graph before you tune the prompt.** Depth is a fact about the task; batching is a fact about the turn.
2. **Batch a level, never a chain.** Put the documented line in your system prompt: *"Only batch tool calls that are independent of each other."*
3. **Return every result of a fan-out in one user message.** Splitting them teaches the model to stop batching.
4. **Pair the health ratio with a wasted-call count.** The ratio alone rewards guessing.
5. **Cache the tool definitions from day one.** It is the documented first step and it costs a breakpoint.
6. **Add tool search when the catalogue passes twenty to thirty tools,** and add it for accuracy first, cost second.
7. **Compute your own crossover** before deferring. Below it, deferring moves definitions from a 0.1× line to a 1.0× line.
8. **Never defer the search tool, and never put `cache_control` on a deferred tool.** Both are 400s.
9. **Pass search blocks back unchanged** and never return a `tool_result` for a `srvtoolu_` id.
10. **Reach for programmatic tool calling when the problem is context or re-sampling,** not when it is the number of network exchanges.
11. **Bound every loop.** `max_iterations` for a cyclical toolset; the graph's depth for an acyclic one.
12. **Catch exceptions inside the tool** and return Chapter 34's instructive string, so the Tool Runner never has to translate one.
13. **Watch the carried share, not the total.** Two-thirds history is the point at which caching and editing start to matter more than anything about your tools.
14. **Change the graph when four turns is too slow.** Consolidation (Chapter 32) is a latency lever, not only a token one.

35.23 Common Mistakes

COMMON MISTAKE

Treating the health ratio as a target

Symptom. A dashboard metric improves and latency does not.

Cause. `total_tool_calls / tool_call_messages` counts calls, including the ones that failed for want of an input. Over-batching maximises it.

Fix. Report the ratio alongside the count of `is_error` results caused by missing or invalid parameters, and treat 1.0 as a floor rather than a goal.

Detection. §35.16: 2.75 for the wrong plan against 1.75 for the right one.

COMMON MISTAKE

Believing programmatic tool calling removes round trips

Symptom. A latency budget built on "one request instead of seven" that does not materialise.

Cause. The sandbox pauses on every call to one of your tools and the API returns a `tool_use` block. Sampling collapses to one; HTTP does not.

Fix. Budget the two separately. Use programmatic calling for context and re-sampling cost, and level batching for exchange count.

Detection. Count `POST`s, not turns. §35.16 shows 1 sampling against 5 HTTP on a seven-node graph.

COMMON MISTAKE

Deferring everything

Symptom. `At least one tool must have defer_loading=false. All tools cannot be deferred.`

Cause. `defer_loading: true` applied uniformly, usually including the tool search tool itself.

Fix. Keep the search tool and the three to five most-used tools loaded.

Detection. `toolsearch.check_request` reproduces the 400 offline; put it in the request builder.

COMMON MISTAKE

Returning a tool_result for the search's id

Symptom. A 400 on the turn after a successful search.

Cause. A loop that emits a result for every block carrying an id, without checking whether the id begins `srvtoolu_`.

Fix. Branch on block type. Chapter 28's reader already separates client and server tool blocks.

Detection. Assert that no outgoing `tool_use_id` matches a server tool id before the request leaves your process.

COMMON MISTAKE

Letting a discovered definition rot in the conversation body

Symptom. Input tokens climb turn after turn on a conversation whose prefix is stable.

Cause. Discovered definitions are expanded inline in the body, and the body is not covered by a prefix-only breakpoint.

Fix. Either move a breakpoint forward so the history is cached, or accept the cost knowingly — §35.18 says which side of the crossover you are on.

Detection. `input_tokens` rising while `cache_read_input_tokens` is flat.

COMMON MISTAKE

Optimising the tools array when the history is the problem

Symptom. Weeks spent shortening descriptions for a saving in the single-digit per cent.

Cause. Two-thirds of the input on a long chain is carried history, and the tools array is in the cached third.

Fix. Measure the carried share first. Then choose from §35.4's table by where the tokens are.

Detection. §35.17: 65.6 per cent carried at thirty turns.

COMMON MISTAKE

Assuming the loop that stopped was yours

Symptom. A response arrives with no answer and no obvious error.

Cause. Three loops are running and only one is yours. `pause_turn` is the server's sampling loop hitting its documented default of ten iterations; a sandbox timeout is the innermost loop; `max_iterations` is yours.

Fix. Branch on `stop_reason` first (Chapter 28), then inspect `stderr` in any `code_execution_tool_result`, then check your own bound.

Detection. Log which of the three bounds was hit, every time a run ends without an answer.

COMMON MISTAKE

Shipping the runner's default error string

Symptom. Internal hostnames and connection strings in transcripts, and models that retry the same failing call.

Cause. The Tool Runner's documented behaviour is to return the exception's type and message. That is Chapter 34's raw arm with two disclosure patterns removed and none of the four criteria gained.

Fix. Catch inside the tool; return an instructive string.

Detection. §35.19: 0/40 actionable and 36/40 leaking, identical to the raw arm.

35.24 Performance Considerations

Latency is depth times a turn, plus the tools. Chapter 11's two-term model applies per turn, and decode dominates. A four-level chain on Opus 5 with 700-token answers spends four decodes; collapsing it to three levels by consolidating two tools is worth more than any prompt change, because it removes a whole decode.

Parallel execution inside a level is your decision. Chapter 31 established that the API does not prescribe execution order. A level of five calls where four take 100 ms and one takes 3 s costs 3 s if you run them concurrently and 3.4 s if you run them in sequence — and the turn does not end until the last result is ready either way, so concurrency inside a level is nearly free latency.

Tool search costs a turn. The documentation frames the trade honestly: *"This trades a small amount of latency (one extra turn to look up a tool) for a large reduction in baseline context usage."* Keeping the three to five most-used tools non-deferred is what stops that turn being spent on every request.

Programmatic tool calling buys decodes, not exchanges. One sampling round trip against four, on the triage graph. If a decode is 3 s and an HTTP exchange with a paused container is 200 ms, the saving is large; if your tools are slow and your decodes are fast, it is small.

Prefill dominates once the history is big. By turn thirty of $35.17's chain, the request carries 663,000 raw tokens in total and roughly 30,000 on the final turn. Chapter 30's caching turns most of that into a 0.1× read and, more importantly for latency, into prefill the server has already done. This is the direct link between the cache configuration and time-to-first-token that Chapter 30 built and this chapter spends.

A number to watch. Carried share. Below a third, tune the tools. Above two-thirds, tune the history. $35.17's chain crosses two-thirds around turn twenty, which is a useful reminder that the right lever changes *during* a conversation.

35.25 Security Considerations

Every level is a new opportunity for injected instructions to act. Chapter 31 established that tool results are untrusted; a chain means a result from level 0 shapes the *arguments* of level 1. The gate that matters is Chapter 34's: values from a tool result are evidence, and a call whose input came from an earlier result deserves the same call-site validation as one the model invented, because in both cases the model chose it.

Over-batching is a security bug as well as a cost bug. A guessed input to a read is a wasted call. A guessed input to a write is Chapter 34's whole subject arriving unannounced. Tools with effects should be reachable only from a level whose inputs are resolved, and the cheapest enforcement is that the effect tool's schema requires an identifier that only an earlier result can supply — plus the call-site check that the identifier exists.

A discovered tool is a tool you did not decide to expose on this turn. Tool search widens the surface at run time by design. Two disciplines follow: the deferred catalogue should contain only tools this *principal* may use, filtered when you build the `tools` array; and a custom client-side tool search is the natural place to apply that filter, because you control what `tool_reference` blocks it returns.

The sandbox runs your tools without the model in the loop. Under programmatic tool calling, your tool is called by code, and the documentation is explicit that tool results are returned as strings that *"can contain any content, including code snippets or executable commands that may be processed by the execution environment."* The gates of Chapter 34 belong on the call site regardless of which loop called it — and `allowed_callers` is how you say which loop is allowed to.

Longer chains disclose more. Each `tool_result` is content that has left your system into the conversation. A thirty-turn chain has thirty of them, all resent on every turn, all potentially in a cache entry. Context editing is a privacy control as well as a cost control.

35.26 Real Production Examples

A five-server operations agent. GitHub, Slack, Sentry, Grafana and Splunk behind one assistant: about 120 tools and roughly 55,000 tokens of definitions. Tool search with BM25, the four most-used tools

non-deferred, namespaced names (`github_`, `slack_`) so one query matches a group, and a system prompt line naming the categories — all four of these are documented optimisation tips, and the last one is the cheapest.

A research assistant. `web_search` plus `web_fetch`, cite-then-fetch: search, inspect the snippets, fetch the two or three that matter. The team's measured win was not tokens but relevance — fetching everything the search returned filled the window with pages the answer did not use.

A batch reconciliation job. Twenty account lookups per run, none dependent on any other. Moved from twenty turns to one code-execution script; sampling round trips fell to one and the intermediate results left the context entirely. The HTTP exchange count did not change, which surprised the team and is exactly what the documentation says.

A coding agent. `text_editor` plus `bash`, a cycle rather than a chain, bounded by `max_iterations` because there is no depth to bound it. The team's incident was a loop that edited and tested for forty iterations on a test that could never pass; the fix was a bound and a reflection step, which is Chapter 51's subject.

A migration that deleted a metric. A team removed `total_tool_calls / tool_call_messages` from their dashboard after discovering it had been rising for a month because a prompt change had started over-batching. They replaced it with the pair — ratio and wasted calls — and the ratio fell back to 1.75.

35.27 Certification Notes

Concept	Where it is examined	Domain
Dependency levels and batching independence	Chain design and orchestration	D1 Agentic Architecture & Orchestration
Sampling versus HTTP round trips	Latency and cost reasoning	D1, D5
Programmatic tool calling and `allowed_callers`	Tool infrastructure	D2 Tool Design & MCP Integration
Tool search variants, limits and rejections	Scaling a tool catalogue	D2
`defer_loading` and prompt caching interaction	Context management	D5 Context Management & Reliability
The four context-management approaches	Choosing a lever	D5
Tool Runner, `max_iterations`, error defaults	SDK usage	D1, D2
Documented tool combinations	Agent patterns	D1
Three nested loops and their bounds	Debugging an agent	D1, D5

CERTIFICATION NOTE

Exam Tips

- **Depth, not count.** A question giving you seven tools and asking for the minimum number of turns is asking for the critical path.
- **Programmatic tool calling collapses sampling, not HTTP.** An option claiming "one request instead of N" is wrong.
- **At least one tool must be non-deferred,** and it must not be the search tool that is deferred. Both are 400s with quotable messages.
- **Regex 200 characters, BM25 500 characters, 5 results by default, 10,000 deferred tools maximum.**
- **Tool search reads names, descriptions, argument names and argument descriptions.** This is why namespacing matters (Chapter 33).
- **Deferred tools are excluded from the prefix; discovered ones expand into the conversation body.** That sentence is the whole caching story.
- **A deferred tool cannot carry** `cache_control`**.**
- **Never return a** `tool_result` **for a** `srvtoolu_` **id;** an empty `tool_references` array is a successful search, not an error.
- **Selection accuracy degrades above 30–50 tools.** Learn the range and the ~55k figure for a five-server setup.
- `max_iterations` **is supported by all seven SDK tool runners.**

35.28 Twenty Practice Questions

1. A task uses nine tools across five dependency levels. What is the minimum number of assistant turns with client tools?
2. Your agent's `total_tool_calls / tool_call_messages` is 1.00. What is wrong?
3. The same ratio is 3.4 and latency is unchanged. What is wrong now?
4. Programmatic tool calling turns a six-call chain into how many sampling round trips, and how many HTTP round trips?
5. Which tokens does programmatic tool calling remove from your bill?
6. Name the four documented approaches to tool-context pressure and what each reduces.
7. What is the documented order in which to adopt them?
8. Which tool search variant takes a Python regular expression, and what is its length limit?
9. What does the API return when a search matches nothing?
10. Where is a discovered tool's definition expanded, and why does that matter for cost?
11. What two request-time 400s does tool search document?
12. What happens if you put `cache_control` on a tool with `defer_loading: true`?
13. Above roughly how many tools does the documentation say selection accuracy degrades?
14. What does the documentation say a five-server MCP setup costs in definition tokens?
15. Which fields does tool search match against?

16. When does the documentation say to use the manual loop rather than the Tool Runner?
17. What does the Tool Runner put in a `tool_result` when your tool raises?
18. What bounds each of the three nested loops in a tool-using system?
19. Which documented tool pairing is a cycle rather than a chain?
20. You must return a `tool_result` for every `tool_use` id. Does that include the tool search tool's block?

35.29 Answers

1. Five. Depth is the floor. *Distractors:* nine confuses node count with depth; one assumes batching removes dependencies; "it depends on the model" is wrong — no model makes a dependent call before its input exists.

2. Under-batching. Every call is in its own turn. The documented ratio *"should be > 1.0"*, and a common cause is returning each result in its own user message, which teaches the model to stop batching. *Distractor:* "nothing, 1.0 is the target" — it is a floor.

3. Over-batching. Dependent calls are being requested alongside the calls that produce their inputs, so the extra calls are guesses that fail and are reissued. Depth is unchanged, which is why latency is unchanged. *Distractor:* "the tools are slow" — that would change latency, not the ratio.

4. One sampling round trip, seven HTTP round trips (one to start plus one per paused tool call, modulo batching inside the script). *Distractor:* "one of each" is the common misreading and the documentation contradicts it directly.

5. The intermediate `tool_result` tokens. *"Tool results from programmatic invocations do not count toward your input/output token usage. Only the final code execution result and Claude's response count." Distractor:* "the tool definitions" — those are unchanged.

6. Tool search reduces definitions loaded upfront; **programmatic tool calling** reduces `tool_result` round trips; **prompt caching** reduces the cost of repeated definitions, not their number; **context editing** removes old `tool_result` blocks from history.

7. Caching from day one; tool search past roughly twenty tools; context editing once conversations run long; programmatic tool calling for repetitive chains of small calls. They compose.

8. `tool_search_tool_regex_20251119`, 200 characters. BM25 takes natural language up to 500. *Distractor:* swapping the two limits.

9. A `tool_search_tool_search_result` with an empty `tool_references` array — a success, not an error. *Distractor:* a 400; *distractor:* a result-level error code. The four codes exist but are for invalid input, unavailability, rate limits and time limits.

10. Inline in the conversation body, not in the prefix. The prefix cache is preserved, but the definition is now in the part of the request that a prefix-only breakpoint does not cover, so it is billed at the full rate every turn. *Distractor:* "in the prefix, after the cache breakpoint" — it is not in the prefix at all.

11. `At least one tool must have defer_loading=false. All tools cannot be deferred.` and `Tool reference '<name>' not found in available tools`.

12. A 400. Put the breakpoint on a non-deferred tool. *Distractor:* "it is ignored" — several caching edge cases are silent, but this one is not.

13. 30 to 50. *Distractor:* 100, or "it does not" — the documentation states the degradation explicitly and it is the main reason to adopt tool search.

14. About 55,000 tokens for GitHub, Slack, Sentry, Grafana and Splunk, *"before Claude does any work"*.

15. Tool names, descriptions, argument names and argument descriptions — both variants. This is why Chapter 33's namespacing advice is called out as *"especially important when using tool search"*.

16. When you need human-in-the-loop approval, custom logging, or conditional execution.

17. The exception's message — in Python its type and message — not the full stack trace, with `is_error: true`. $35.19 scores it identically to a raw exception: 0/40 actionable, 36/40 leaking. *Distractor:* "a generic failure message"; *distractor:* "the full traceback".

18. Your loop: a non-`tool_use` stop reason, or `max_iterations`. **The server's loop:** a documented default of ten iterations per request, surfaced as `pause_turn`. **The sandbox's loop:** the script returning, with a container idle limit of 4.5 minutes and a 30-day maximum.

19. The coding agent, `text_editor` + `bash` — inspect, edit, test, repeat. It has no depth, which is why it needs an iteration bound.

20. No. Its block is a `server_tool_use` with a `srvtoolu_` id; returning a `tool_result` for it causes the API to reject the request. Pass the search blocks back unchanged.

35.30 Ten Practical Exercises

1. Add a node to `CHAIN` that depends on `post_summary` and re-run the planners. Predict the new depth before you run it.
2. Add latency weights to `Plan` and compute wall-clock for each strategy under sequential and concurrent execution within a level.
3. Extend `all_at_once` to model a graph where level 0 is a single node, and report how the wasted-call count changes.
4. Sweep `TOOL_RESULT` in `ctxgrow.py` from 200 to 4,000 tokens and find where the carried share crosses two-thirds.
5. Add a fifth regime combining a rolling message cache *and* context editing, and report whether the savings multiply.
6. Re-run `crossover` with your own measured `tokens_per_tool`, `hot` and `discovered`, and state which side of it you are on.
7. Model a catalogue where 20 of 120 tools are used every turn, and decide whether they should be deferred.
8. Add `tool_search_tool_result_error` fixtures for all four error codes and make your reader handle each distinctly.
9. Write the request builder that calls `toolsearch.check_request` and `check_continuation` before every send, and add a test for each of the six rejections.

10. Score your own five worst error strings with `runnerarm` and rewrite the worst one.

35.31 Five Challenge Labs

1. **A real planner.** Extend `chainplan` to accept per-node latency and cost, and emit the schedule minimising wall clock subject to the dependency constraints. Compare it against level batching on a graph with one very slow node, and report whether the optimum ever differs.
2. **The crossover, measured.** Build two identical agents differing only in `defer_loading`, run 200 matched requests through each with `count_tokens` rather than live calls, and publish the measured crossover with an interval against §35.18's computed one.
3. **A filtered custom search.** Implement client-side tool search that returns `tool_reference` blocks only for tools the current principal may call, and demonstrate that a deferred tool outside the grant is never discoverable. State what this does and does not prevent.
4. **Three loops, one trace.** Instrument a run that uses a client tool, a server tool and a code-execution script, and produce a single trace showing which loop was active at each point and which bound ended each. This is Chapter 57 in miniature.
5. **Break the chain honestly.** Take your longest chain and find a consolidation (Chapter 32) that removes one level. Measure latency and accuracy before and after, paired, with an interval — and publish the result even if the accuracy fell.

35.32 Ten Interview Questions

1. Seven tools, four levels. How many turns, and what would change that?
2. What is the difference between a sampling round trip and an HTTP round trip, and which does programmatic tool calling reduce?
3. Your health ratio went up and latency did not. Diagnose it.
4. When would you *not* enable tool search?
5. Where does a discovered tool definition live, and what does that cost?
6. Name the four context-management approaches and say which one you would reach for first.
7. What are the three loops in a tool-using system and what bounds each?
8. What does the SDK Tool Runner do with an exception, and why does that matter?
9. How would you make an over-batched agent stop guessing inputs?
10. Your agent has 400 tools. Walk me through the design.

35.33 Summary

Part 6 began with a single `tool_use` block and ends with a graph. Chapter 31 established the contract — the model proposes, only your loop disposes — and the continuation rules that make a follow-up request legal. Chapter 32 turned the schema into a budget and measured what `strict: true` does and does not enforce. Chapter 33 established that the description is the whole interface and measured a catalogue's confusability rather than asserting it. Chapter 34 put five gates between the model's request and your effect. This chapter asked what happens when there are forty of these instead of one, and the answer

turned out to be mostly about shape.

The shape is a dependency graph, and its depth is the round-trip floor. Seven nodes, four levels: no prompt makes it three, and the only way below the floor is a different graph, which is Chapter 32's consolidation argument arriving as a latency lever. Between the floor and the ceiling sit four strategies, and the chapter's first negative result is that the documented health metric cannot tell two of them apart in the right order. Over-batching scores 2.75 against the correct plan's 1.75, because a guessed call is still a call. The ratio detects the failure it was published for — under-batching, which scores exactly 1.00 — and is blind to the other by construction. Pair it with a wasted-call count or it will reward guessing, and on a tool with an effect a guessed input is Chapter 34's subject arriving unannounced.

The second thing worth carrying is that *round trip* is two quantities. Programmatic tool calling collapses sampling round trips from seven to one and leaves the HTTP exchanges alone, because the sandbox pauses on every call to one of your tools; what it removes instead is the intermediate results, which the documentation says do not count toward your token usage at all. Which of those numbers matters depends on whether your problem is decode latency or network exchanges, and answering that question first is most of the design.

Then the money, which on a long chain is not where people look for it. Two-thirds of the raw input across a thirty-turn chain is carried history rather than prefix, so the tools array — the thing everyone optimises — is in the smaller third and is already cached. Four documented approaches target four different sources: tool search the definitions, programmatic calling the result round trips, caching the cost of repetition, context editing the history itself. Priced against the same chain they are worth 4.31×, 2.21× and 8.09× respectively, they compose, and only context editing changes the raw token count rather than the bill. Chapter 16's statelessness multiplier reappears here as 2.57× rather than 26.2×, which is the useful reminder that it was always a property of a token distribution and never a constant.

The chapter's second negative result is about the feature everyone believes is free. Tool search is documented as cutting a 55,000-token five-server catalogue by over 85 per cent, and the mini project reproduces a 71 per cent per-turn saving at that scale. But a deferred definition is excluded from the *prefix* and a discovered one is expanded into the *conversation body*, which moves it from a 0.1× line to a 1.0× line, and below about 35 tools under prefix-only caching that makes deferring more expensive — 3.5× more at ten tools. The crossover lands next to the documented accuracy threshold of 30 to 50 tools, and the accuracy reason is the one that should decide it, because no cache configuration fixes selection. Finally, the SDK Tool Runner's documented default — the exception's type and message, without the traceback — scores 0 out of 40 actionable and 36 out of 40 leaking on Chapter 34's scorer, identical to a raw exception, because removing a stack trace removes the patterns that live in a stack trace and none of the four things a retry needs. Catch inside the tool. Part 7 opens next, with tools whose schemas, descriptions and error strings someone else wrote.

KEY TAKEAWAYS

1. **A chain is a graph; its depth is your round-trip floor.** Seven nodes, four levels, and no prompt changes that.
2. **Batch a level, never a chain.** Independent calls in one turn; dependent calls in the next.
3. **The documented health ratio is a floor, not a target** — over-batching scores 2.75 against the correct plan's 1.75.
4. **Sampling round trips and HTTP round trips are different numbers,** and programmatic tool calling collapses only the first.
5. **Two-thirds of a long chain's input is carried history.** Optimise where the tokens are.
6. **The four context levers target four different sources** and compose; only context editing shrinks the window rather than the bill.
7. **Deferred tools leave the prefix; discovered tools enter the conversation body.** That one sentence is the whole caching story.
8. **Compute your own crossover.** Below roughly 35 tools with prefix-only caching, deferring costs more than caching.
9. **Three loops run, and only the outermost executes your client tools.** Know what bounds each before you debug one.
10. **The Tool Runner's default error string is a diagnosis, not an instruction.** Catch inside the tool and write the sentence yourself.

35.34 Glossary

Term	Meaning
BM25 variant	`tool_search_tool_bm25_20251119`; natural-language queries up to 500 characters
Chain	A set of tool calls where at least one input depends on another's output; a DAG, not a list
Context editing	Removing stale `tool_result` blocks from history; the only lever that shrinks the window itself
Crossover	The catalogue size at which deferring becomes cheaper than caching; ~35 tools with prefix-only caching
`defer_loading`	Keeps a tool out of the system-prompt prefix until discovered; the definition is still sent every request
Depth	Number of dependency levels; the minimum number of assistant turns
HTTP round trip	One `POST /v1/messages`, which may return a paused container rather than a generation
Level	A set of nodes whose dependencies are satisfied; the correct unit of a fan-out
`max_iterations`	The Tool Runner's bound on the outer loop; supported by all seven SDKs
Over-batching	Requesting a dependent call in the same turn as the call producing its input
Programmatic tool calling	Chaining inside a code-execution sandbox; one sampling round trip, results out of context

Term	Meaning
Regex variant	`tool_search_tool_regex_20251119`; Python `re.search` patterns up to 200 characters, case-insensitive
Sampling round trip	One generation; the unit Chapter 11's latency model charges for
Tool search	Server-side discovery of deferred tools, returning `tool_reference` blocks
`tool_reference`	A block naming a discovered tool; expanded by the API into a full definition
`tool_search_tool_result`	The server-side search result block; a custom client-side search uses an ordinary `tool_result` instead
Under-batching	Splitting independent calls across turns; legal, and documented as teaching the model to stop batching

35.35 Further Reading

Primary sources, verified 4 August 2026 at `platform.claude.com/docs/en`:

- `agents-and-tools/tool-use/manage-tool-context` — the four approaches, what each reduces, and the documented adoption order.
- `agents-and-tools/tool-use/tool-search-tool` — both variants, `defer_loading`, the response blocks, the continuation rules, the limits, the six rejections, the custom client-side implementation, and the ~55k and 30–50 figures.
- `agents-and-tools/tool-use/programmatic-tool-calling` — the pause-and-resume model, `allowed_callers`, the container lifecycle, and the statement that programmatic tool results do not count toward token usage.
- `agents-and-tools/tool-use/tool-combinations` — the five documented pairings.
- `agents-and-tools/tool-use/tool-runner` — the SDK loop, `max_iterations`, the exception-to-`is_error` behaviour, and when to use the manual loop instead.
- `agents-and-tools/tool-use/parallel-tool-use` — the batching rules and the health metric, verified in Chapter 31.
- Anthropic Engineering, *Advanced tool use* and *Effective context engineering for AI agents* — the essays the tool search page points to for the scaling argument and for just-in-time retrieval.

Where to go next in this book. Chapter 36 opens Part 7 with the Model Context Protocol, where the tools arrive from servers you did not write and every gate in Chapters 34 and 35 becomes load-bearing. **Chapter 49** takes planning and replanning as a subject in its own right, and **Chapter 50** covers orchestrator–subagent topologies, which is what happens when a level becomes a whole agent. **Chapter 52** builds the context budget this chapter measured, **Chapter 53** covers the retrieval that a custom tool search is an instance of, and **Chapter 54** covers compaction and memory. **Chapter 57** turns the four counts of $35.13 into a trace.